

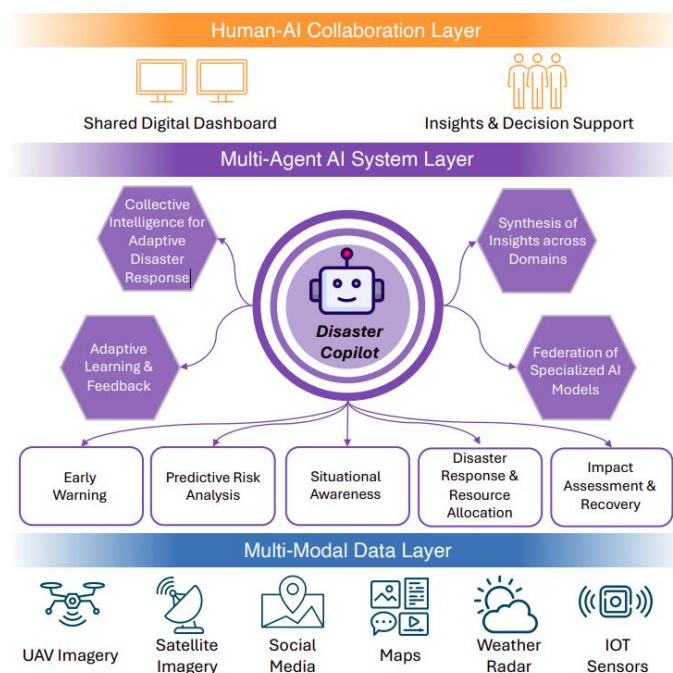
1. Disaster Copilot

Reference: Disaster Management in the Era of Agentic AI Systems, [arXiv:2510.16034](https://arxiv.org/abs/2510.16034)

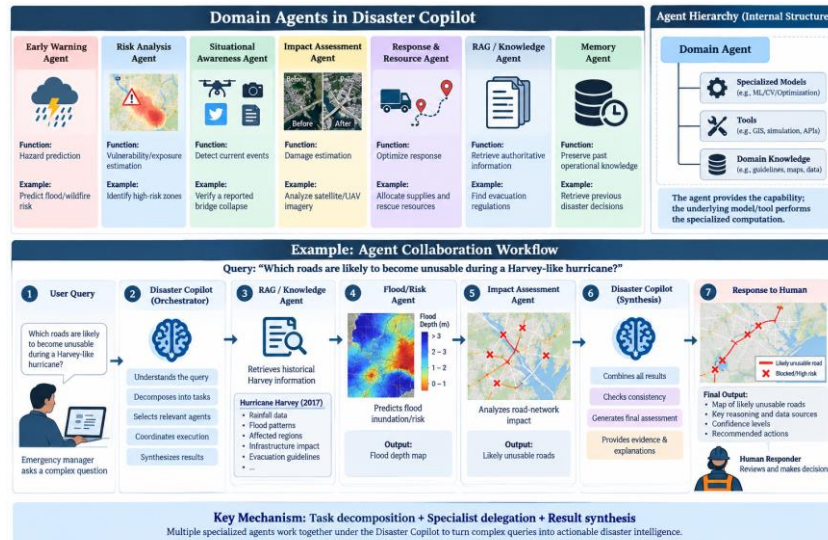
Abstract: Disaster management relies on many specialized AI systems (forecasting, risk assessment, damage detection, evacuation planning, resource allocation), but these systems are fragmented — different data formats, resolutions, and interfaces — forcing emergency managers to integrate outputs manually. Disaster Copilot proposes a central multimodal LLM orchestrator that decomposes disaster-management objectives, delegates tasks to domain-specific sub-agents, and synthesizes their outputs into one operational picture. It also incorporates RAG, persistent memory, edge/on-device operation, multilingual interaction, and human-AI collaboration. **This is a system-level architectural vision and research roadmap, not an experimentally validated deployment.**

Core problem: Fragmented AI → fragmented intelligence → manual coordination.

Architecture:



Workflow:



See attached poster image — it shows the seven domain agents (Early Warning, Risk Analysis, Situational Awareness, Impact Assessment, Response & Resource, RAG/Knowledge, Memory), the internal agent hierarchy (specialized models + tools + domain knowledge), and a full 7-step worked example query.

Key Features: Multimodal input fusion · RAG-based knowledge retrieval · persistent institutional memory · edge AI for low-connectivity operation · human-in-the-loop oversight · digital twin integration for scenario simulation.

Limitations:

- Vision/architecture paper — not a validated end-to-end deployment
- Errors can propagate between dependent agents
- Heterogeneous models create interoperability problems
- Models may fail under unseen disaster conditions
- Real-time multi-agent orchestration can introduce latency

Research Scope (open problems this paper identifies):

1. Reliable agent orchestration — dynamic agent selection, task planning, execution monitoring

2. Evidence-aware disaster RAG — hybrid retrieval with provenance, confidence, temporal/geospatial awareness
3. Cross-agent communication — standard schemas (prediction, location, timestamp, confidence, provenance)
4. Agent conflict resolution — detecting and reconciling contradictory agent outputs

2. CrisiSense-RAG

Reference: CrisiSense-RAG: Crisis Sensing Multimodal Retrieval-Augmented Generation for Rapid Disaster Impact Assessment

Abstract: CrisiSense-RAG is a multimodal RAG system for rapid disaster impact assessment. Its core insight: different sources capture a disaster at different times — social media and 311 emergency calls report conditions *during* the event (e.g., peak flooding), while aerial imagery is often collected *after* (e.g., once water has receded). Rather than treating all evidence equally, the system analyzes text and images separately, then combines them with **time-aware fusion**. Tested across four LLM backends (Gemini 2.5 Flash, Gemini 3 Flash, Qwen 3.5, GPT-5-mini) on 110 Houston ZIP codes, the system achieved flood-extent error (MAE) of 8.86–19.64% and damage-severity error of 10.14–14.47%, with multimodal fusion cutting flood-extent error by up to 8.62 percentage points compared to text-only. Code and dataset are both publicly released (GitHub and Hugging Face).

Architecture:

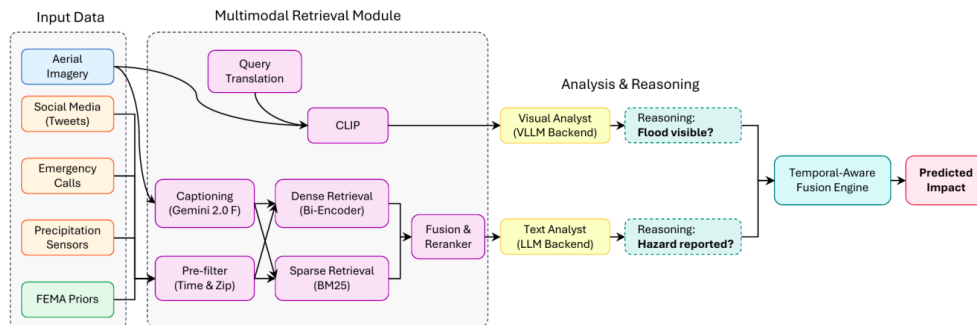


Figure 1: Overview of the proposed CrisiSense-RAG architecture.

How it works (Retrieve → Analyze → Fuse → Assess):

- **Retrieve:** Twitter/X, Houston 311 reports, rainfall sensors, NOAA aerial imagery, historical FEMA loss data — using dense + BM25 retrieval with reranking for text, geographic containment + CLIP reranking for images.
- **Analyze separately:** a Text Analyst estimates impact from social/311/rainfall data; a Visual Analyst examines aerial imagery for flooding, debris, and structural damage.
- **Fuse with time-awareness:** flood extent weights text ~75–80% over visual ~20–25%, since text captures peak conditions that later imagery may miss; damage severity uses text as baseline, adjusted upward when imagery shows persistent structural damage.
- **Assess:** produces quantitative flood extent and damage severity, each linked to its supporting evidence.

How it compares to other methods:

CrisiSense-RAG was benchmarked against four published flood-damage-estimation methods, including supervised models trained on labeled data (e.g., an insurance-claims-based ensemble, a LightGBM model). Despite using no task-specific training, CrisiSense-RAG's best result (10.09% MAE) was competitive with these supervised systems — a notable result since it means the zero-shot approach doesn't sacrifice much accuracy for the flexibility of not needing labeled training data.

Limitations:

- Evaluated only on Hurricane Harvey/Houston — generalization to other disasters (earthquakes, wildfires, tornadoes) and regions unverified
- ZIP-code-level granularity — no building-level precision
- Confidence scores are not statistically calibrated
- Does not address conflicting or intentionally misleading reports
- Validated on historical data, not a live real-time stream

3. Side-by-Side Comparison

	Disaster Copilot	CrisiSense-RAG
Scope	Full multi-agent orchestration architecture	Single capability: impact assessment

Validation status	Conceptual / architectural vision only	Empirically tested on Hurricane Harvey
Feasibility as a final-year project	Very large — a systems-research program	Achievable — narrow, measurable scope
Key limitation	No working implementation to build on	Narrow generalization (one disaster, one region)
Role in a combined system	Orchestration layer (decides what's needed)	Impact-analysis layer (tells Copilot what happened)

How they relate: CrisiSense-RAG can function as the *Impact Assessment Agent* inside Disaster Copilot's orchestration layer:

Disaster Copilot → Orchestrator → Impact Agent → CrisiSense-RAG → Text + Images → Temporal Fusion → Flood & Damage Assessment

In short: Copilot decides *which* capability is needed; CrisiSense determines *what actually happened*. They are complementary, not competing.

4. Final verdict:

CrisiSense-RAG is the more realistic foundation for this project — its scope and evaluation protocol are narrow enough to produce defensible, measurable results. A full Disaster Copilot is too large a systems-research program for a single project. **The stronger path is to build a CrisiSense-like validated impact-assessment core, then add a constrained orchestration layer around it** — framed as "an agentic disaster decision-support architecture incorporating temporally aware multimodal impact assessment," rather than attempting a fully autonomous Copilot.

The underlying research gap both papers point to is the pipeline:

"What happened?" → "What does the evidence imply?" → "What should the system do next?" → "Did the action change the situation?"

CrisiSense addresses the first two stages; Copilot addresses the architecture for the rest. Closing this loop fully — with explicit uncertainty, evidence traceability, and human approval — is more ambitious than either paper alone, but a scaled-down version of it is a viable project target.